

A PROJECT REPORT ON

LLM based test case generation from image of form

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Aditya Dhodade
Prasad Dhondge
Vedant Rawat
Siddhesh Dhadse

Exam No: B400050377
Exam No: B400050378
Exam No: B400050373
Exam No: B400050565



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled
LLM based test case generation from image of form

Submitted by

Aditya Dhodade
Prasad Dhondge
Vedant Rawat
Siddhesh Dhadse

Exam No: B400050377
Exam No: B400050378
Exam No: B400050373
Exam No: B400050565

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. U.S. Pawar** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. U.S. Pawar
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. P. T. Kulkarni
Principal
Pune Institute of Computer Technology

Place:Pune
Date:

BE-Project Sponsorship Letter

Date : 13-09-2024
Place - Sarvatra Tech. Pune

Whomsoever it may concern

This is to inform that the following students of Pune Institute of Computer Technology (PICT), Pune (Engineering branch: Computer Engineering) have been selected for executing sponsored BE project at eLab Sarvatra Technologies for the annual period of 2024-2025.

1. Aditya Dhodade
2. Prasad Dhondge
3. Vedant Rawat
4. Siddhesh Dhadse

Title of project: **Multimodal LLM 2 - AI/LLM based test case generation from image of form**

The management authorizes the above students to present this project as their Final Year project.

This authorization is only for academic purposes and no student can claim the intellectual property of the developed solution for any unauthorized/commercial use.

Authorised Signatory

Manju Jacob.

Sr. VP - Human Resource.

Sarvatra Technologies Pvt. Ltd.

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**LLM based test case generation from the image of form**’.*

*We would like to take this opportunity to thank **Prof. U. S. Pawar** giving me all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

*We are also grateful to **Dr G.V. Kale**, Head of Computer Engineering Department, PICT for his indispensable support, suggestions.*

*In the end our special thanks to **Nikhil Karkhanis**, Sarvatra Technologies and all faculty members for their whole hearted cooperation for completion of this report. We also thank our laboratory assistants for their valuable help in laboratory.*

Aditya Dhodade
Prasad Dhondge
Vedant Rawat
Siddhesh Dhadse
(B.E. Computer Engg.)

ABSTRACT

Software testing is a crucial part of the development cycle, ensuring that applications meet user expectations and function without errors. However, generating exhaustive test cases is a time-consuming and often manual process, especially when it involves forms with complex structures and varied input fields. Traditional methods of test case generation, particularly for form-based applications, rely on manual interpretation of form layouts and input fields which is prone to human error and limits the number of test cases that can be generated within given time constraints. OCR (Optical Character Recognition) methods which are used to extract text from form images struggle with document structure and layout. Although some machine learning models assist in the testing process, they lack the ability to automatically understand the complex interactions between the form's layout and input fields. With advancements in artificial intelligence (AI) and large language models (LLMs), there is an opportunity to automate this process using multimodal generative models. Our approach uses different generative models to analyze the subject to be tested and generate test cases to find the loophole in the problem. Further these test cases can be tested automatically. This approach significantly improves test coverage, reduces manual effort, and accelerates the testing process, ensuring robust software performance with minimal human intervention.

Contents

Abbreviations	12
1 Introduction	1
1.1 Overview	2
1.2 Motivation	2
1.3 Problem Definition and Objectives	2
1.4 Project Scope & Limitations	3
1.5 Methodologies of Problem solving	3
2 Literature Survey	4
2.1 Traditional UI Testing Tools	5
2.2 Automated UI Testing	5
2.3 Computer Vision in UI Testing	5
2.4 Vision-Language Models for UI Understanding	6
2.5 Natural Language Code Generation and Test Scripting	6
2.6 Comparative Studies on LLM-Based Testing Tools	7
2.7 Gaps in Literature	7
2.8 Research Gap Addressed	7
3 Software Requirements Specification	9
3.1 Assumptions and Dependencies	10
3.2 Functional Requirements	11
3.2.1 System Feature 1: UI Screenshot Upload and Preprocessing	11
3.2.2 System Feature 2: Visual Language Model Processing	11
3.2.3 System Feature 3: Test Case Generation	11
3.2.4 Test Case Execution via Automation Tools	12
3.3 External Interface Requirements	13
3.3.1 User Interfaces	13
3.3.2 Hardware Interfaces	13
3.3.3 Software Interfaces	13
3.3.4 Communication Interfaces	13
3.4 Nonfunctional Requirements	14

3.4.1	Performance Requirements	14
3.4.2	Safety Requirements	14
3.4.3	Security Requirements	14
3.4.4	Software Quality Attributes	14
3.5	System Requirements	14
3.5.1	3.5.1 Database Requirements	14
3.5.2	3.5.2 Software Requirements	15
3.5.3	Hardware Requirements	15
3.6	Analysis Models: SDLC Model to be applied	15
3.6.1	Requirements Analysis Phase	15
3.6.2	System Design Phase	16
3.6.3	Module Design Phase	16
3.6.4	Coding and Integration Phase	16
3.6.5	Testing Phase (Verification)	17
3.6.6	Deployment and Maintenance	17
4	System Design	19
4.1	System Architecture	20
4.2	Data Flow Diagrams	21
4.3	UML Diagrams	25
4.4	Sequence Diagram	26
5	Project Plan	27
5.1	Project Estimate	28
5.1.1	Reconciled Estimates	28
5.1.2	Project Resources	29
5.2	Risk Management	30
5.2.1	Risk Identification	30
5.2.2	Risk Analysis	30
5.2.3	Overview of Risk Mitigation, Monitoring, Management	31
5.3	Project Schedule	32
5.3.1	Project Task Set	32
5.3.2	Timeline Chart	32
5.4	Team Organization	32
5.4.1	Team structure	32
5.4.2	Management reporting and communication	33
6	Project Implementation	34
6.1	Overview of Project Modules	35
6.2	Tools and Technologies Used	35
6.3	Algorithm Details	35

6.3.1	Computer Vision (VLM)	35
6.3.2	Natural Language Processing (LLM)	36
6.3.3	Test Script Synthesis	36
7	Software Testing	37
7.1	Type of Testing	38
7.1.1	Unit Testing	38
7.1.2	Integration Testing	38
7.1.3	System Testing	38
7.1.4	Validation Testing	39
7.2	Test cases & Test Results	39
7.2.1	Test Case Relevance to Uploaded Screenshot	39
7.2.2	Execution Success Rate and Required Modifications	39
7.2.3	Reduction in Human Effort and Time	40
8	Results	41
8.1	Outcomes	42
8.2	Screen Shots	42
9	Conclusions	45
9.1	Conclusion	46
9.2	Future Work	46
9.3	Applications	46
10	Appendix	47
10.1	Appendix A	48
10.2	Appendix B	49
10.2.1	Survey Paper Details:	49
10.2.2	Implementation Paper Details:	49
10.3	Appendix C	50
11	References	51

List of Figures

3.1 V-Model Applied to Intelligent UI Testing Platform Development .	18
4.1 Architecural Diagram	20
4.2 Level 0 Data Flow Diagram	21
4.3 Level 1 Data Flow Diagram	22
4.4 Level 2 Data Flow Diagram	23
4.5 Activity Diagram	25
4.6 Sequence Diagram	26
5.1 Timeline Chart	32
8.1 Streamlit UI	42
8.2 Generated Test Cases	43
8.3 Execution Of Test Case	43
8.4 Time Delay Introduced	44
8.5 XPathS Extracted	44

List of Tables

2.1	Comparison of UI Testing Approaches	7
7.1	Relevance of Generated Test Cases	39
7.2	Test Case Execution and Required Changes (OrangeHRM)	40

Abbreviations

LLM	:	Large Language Model
VLM	:	Visual Language Model
CAPTCHA	:	Completely Automated Public Turing test to tell Computers
and Humans Apart		
URL	:	Uniform Resource Locator
UML	:	Unified Modeling Language
GPT	:	Generative Pre-trained Transformer
LLaMA	:	Large Language Model Meta AI
GUI	:	Graphical User Interface
BDD	:	Behavior-Driven Development
SDLC	:	Software Development Life Cycle
DOM	:	Document Object Model
GPU	:	Graphics Processing Unit

CHAPTER 1

INTRODUCTION

1.1 Overview

UI testing is essential for ensuring the functionality and reliability of modern applications, but traditional methods are often manual, time-consuming, and require technical expertise. Pixel2Test aims to address these challenges by introducing an AI-powered framework that automates UI test case generation using screenshots or webpage URLs.

The system leverages Visual Language Models (VLMs) to analyze UI images and identify interactive elements, and Large Language Models (LLMs) to generate structured, executable test scripts. Through a simple Streamlit interface, users can upload UI screenshots, review auto-generated test cases, and run them using the Behave framework.

By reducing manual scripting and supporting multi-step workflows, this approach accelerates the testing process, improves coverage, and makes UI testing accessible to both technical and non-technical users.

1.2 Motivation

The motivation for this project stems from the growing demand for seamless and reliable user interfaces (UI) in software applications. With digital platforms becoming an integral part of our daily lives, ensuring that users experience smooth and intuitive interactions is crucial for maintaining high user satisfaction and business success. However, traditional UI testing methods, particularly manual testing, are time-consuming, labor-intensive, and often struggle to meet the demands of today's fast-paced development cycles. Automated testing tools, while useful, still face limitations when dealing with dynamic and complex UIs. Therefore, leveraging advancements in generative AI and Visual Language Models (VLMs) presents an opportunity to revolutionize UI testing by automatically generating diverse test cases, ensuring comprehensive coverage, and reducing the manual effort required.

1.3 Problem Definition and Objectives

The current approaches to UI testing, particularly for complex and dynamic user interfaces, are either manual, which is inefficient and error-prone, or automated through tools like Selenium, which can only execute predefined test cases. These methods lack the ability to understand the intricate interactions between UI components, such as form layouts and input fields. Additionally, they struggle with automatically generating exhaustive and diverse test cases that account for both structural and semantic variations. This results in incomplete test coverage and

slower release cycles. The project aims to address this problem by using Visual Language Models (VLMs) to analyze UI form images, identify key elements, and automatically generate comprehensive test cases, improving both the speed and accuracy of UI testing.

1.4 Project Scope & Limitations

The scope of this project includes the automation of UI test case generation using screenshots or webpage URLs as input. The system supports single as well as multi-image inputs to test multi-step user workflows. It leverages advanced AI models—Visual Language Models (VLMs) for UI understanding and Large Language Models (LLMs) for code generation—to create structured test cases and executable scripts in the Behave framework. The entire process is integrated into a user-friendly Streamlit interface, making it accessible to both technical and non-technical users without the need for manual scripting.

However, the system currently focuses on static, form-based user interfaces and may not perform accurately on highly dynamic or interactive UIs. Validation logic for special cases like CAPTCHAs or real-time field constraints is not yet implemented.

1.5 Methodologies of Problem solving

The system begins by accepting either a UI screenshot or a webpage URL as input through a simple Streamlit interface. Once the input is received, a Vision Language Model (VLM) is used to analyze the visual structure of the UI. This model identifies key interactive components like input fields, buttons, labels, and checkboxes, along with their semantic relationships.

Based on the understanding of the UI layout, a Large Language Model (LLM) then generates relevant test cases in a structured JSON format. Each test case includes the test ID, a description, steps to perform the test, and the expected outcome. In parallel, ChromeDriver is used to extract the XPath selectors for the detected elements from a live webpage, enabling precise interaction.

These test cases are then automatically converted into .feature files along with Python step definition scripts that follow the Behave (BDD) framework. The system executes these scripts and displays the test results within the same interface.

CHAPTER 2

LITERATURE SURVEY

2.1 Traditional UI Testing Tools

Early automated UI testing relied on scripting frameworks such as Selenium [1], Cypress [2], and Appium [3]. These tools require manual test script creation using XPath or CSS selectors, which are brittle and require maintenance when the UI changes. Record-and-playback tools like Katalon Studio [4] and TestComplete [5] reduce scripting effort but still suffer from flakiness due to dynamic element locators. As observed in Shi and Zhou [6], even modern automation frameworks demand significant human input and do not scale well in dynamic web environments.

Recent improvements include self-healing locators (e.g., Testim [7], Mabl [8]), which use machine learning to adapt to UI changes. However, these tools still depend on pre-recorded test flows and struggle with entirely new UI structures. Most fall short of fully automating the test generation process from raw visual or natural language inputs. Furthermore, most do not support intelligent reasoning over GUI flows involving multiple pages.

2.2 Automated UI Testing

Machine learning has been applied to automate test case generation:

- **TestGPT** [9] uses GPT models to generate test scripts from natural language descriptions.
- **Diffblue** [10] employs reinforcement learning for unit test generation.
- **TESTAR** [11] automates GUI testing through computer vision and reinforcement learning.

While these approaches reduce manual effort, they primarily focus on API or unit testing rather than end-to-end UI validation.

2.3 Computer Vision in UI Testing

Computer vision techniques have been used to improve test robustness:

- **SikuliX** [12] performs image-based UI testing by matching screenshots.
- **EyeAutomate** [13] uses OCR and template matching for test automation.
- **Applitools** [14] applies visual AI for regression testing.

However, these methods lack semantic understanding of UI elements, making them sensitive to minor visual changes.

2.4 Vision-Language Models for UI Understanding

Vision-Language Models (VLMs) such as BLIP-2, Flamingo, and the LLaMA-3 vision variants have demonstrated significant capabilities in image captioning, visual question answering, and scene interpretation. Their application in software testing is still emerging. Wang et al. [15] explored the use of large VLMs to analyze web-based GUIs for generating automated test scenarios. Their study found that models like GPT-4V can detect buttons, input fields, and layout hierarchies with high accuracy.

Similarly, Van Dam and Serbina [16] investigated VLMs in the context of generating structured descriptions of visual content, albeit in a biological domain. Their approach to taxonomy generation from images aligns conceptually with the way our system generates test case tables from UI screenshots.

However, existing work has primarily focused on static analysis or classification rather than complete UI-to-test automation pipelines. Our project extends the role of VLMs by not only identifying UI components but also feeding their semantic understanding into an automated test case generator.

2.5 Natural Language Code Generation and Test Scripting

Large Language Models (LLMs) like GPT-4, Claude, PaLM, and LLaMA-3 have been widely explored for generating code from text descriptions. Zhang et al. [17] proposed GPT-Test, a framework for generating test scripts directly from screenshots by using multi-modal prompting. Their results showcased LLMs' effectiveness in producing near-executable code templates.

Other efforts, such as LLM4Test by Shi and Zhou [6], focused on generating .feature files for Cucumber/Behave from user stories or functional requirements. Nguyen and Nguyen [18] extended this by combining visual inputs and NLP into a unified assistant for testers. These approaches are highly promising but typically depend on well-structured input. Our approach, in contrast, leverages both the vision and language capabilities of LLaMA-3 to generate structured test cases even from unannotated screenshots and web URLs.

Moreover, our system introduces an intermediate test case table to let users select and refine their intended flow before code generation, reducing hallucination risks from LLMs and enhancing control.

Table 2.1: Comparison of UI Testing Approaches

Approach	Test Generation	Locator Stability	No-Code UI	BDD Integration
Selenium/Cypress	Manual	Low (XPath/CSS)	No	Partial
Testim/Mabl	AI-assisted	Medium (Self-healing)	Yes	No
SikuliX/Apltools	Image-based	High (Visual)	Yes	No
TestGPT/Diffblue	LLM-generated	N/A (API/Unit tests)	No	No
Our Solution	VLM + LLM	High (Dynamic XPath)	Yes	Full

2.6 Comparative Studies on LLM-Based Testing Tools

Recent empirical studies have started to benchmark various LLM-based and ML-integrated UI testing tools. Ran et al. introduced Guardian, a runtime framework that evaluates LLM-based testing agents in live web environments, comparing them against reinforcement learning (RL)-driven agents for UI exploration tasks [19].

Li et al. proposed Test-Agent, a multimodal test generation tool that combines LLMs and deep learning for Android testing. Their framework demonstrated high scalability for mobile GUIs and showed promise in generating complete test flows from screenshots [20].

An extensive study by Garousi et al. offered a systematic review of commercial and academic LLM-driven testing tools, emphasizing the gap in visual and real-world integration across tools like CodiumAI and AutoTest [21].

Ma et al. conducted the first large-scale empirical study comparing how different LLMs perform in web-form test generation. Their results underline that prompt engineering and domain context greatly influence the effectiveness of generated test scripts [22].

2.7 Gaps in Literature

While recent research has independently explored VLMs for GUI understanding and LLMs for code generation, few systems unify these technologies into a coherent, user-centric testing pipeline. Current tools lack the ability to process and align visual workflows across multiple screenshots, dynamically extract live element locators, and generate working code ready for execution.

2.8 Research Gap Addressed

Our work bridges the gap between:

- Visual UI understanding (via VLMs) and test automation.
- Dynamic XPath extraction and BDD script generation.
- End-to-end automation from screenshots to executable tests.

Unlike prior solutions, our framework requires no manual scripting, adapts to UI changes, and supports multi-image workflow testing.

CHAPTER 3

**SOFTWARE REQUIREMENTS
SPECIFICATION**

The Software Requirements Specification (SRS) for the project outlines the functional and non-functional requirements for automating the generation of test cases from UI screenshots of online payment systems. This document serves as a blueprint for the development, testing, and deployment of the AI-driven system that will streamline the UI testing process and ensure efficient, accurate, and thorough testing.

3.1 Assumptions and Dependencies

- **Assumptions:**

- The UI screenshots provided are clear and contain all necessary form elements.
- The Visual Language Model will correctly identify the elements in the screenshot and map them to appropriate test cases.
- The tester has access to an environment where the generated test cases can be run using an automation tool.
- The web pages provided via URL are publicly accessible and loadable by ChromeDriver without authentication.
- The local system has access permissions required to run Selenium, Streamlit, and Behave.

- **Dependencies:**

- The system is dependent on accurate performance of the Visual Language Model (such as LayoutLM or similar models).
- The project assumes the availability of suitable test automation tools (like Selenium) to run the generated test cases.
- The system requires a functioning Python environment (version 3.10 or higher) with essential libraries (e.g., Streamlit, Selenium, Behave) pre-installed.
- The system assumes that users will run the platform in an environment where sufficient computational resources (RAM and optionally GPU) are available to load and run the models efficiently.
- It assumes the operating system supports required browser automation tools (i.e., Google Chrome and ChromeDriver).

3.2 Functional Requirements

This section outlines the functional requirements of the system, detailing the core features that the system will deliver to meet the project objectives. These functional requirements describe how the system should behave and the actions it will perform to achieve desired outcomes.

3.2.1 System Feature 1: UI Screenshot Upload and Preprocessing

The system must allow users to upload screenshots of online forms. The system will preprocess the uploaded images to ensure they are in the correct format for analysis by the Visual Language Model.

- The system must support various image formats (e.g., PNG, JPEG).
- The system must be able to process forms with multiple pages.
- The system must preprocess the images and prepare them for analysis by the model.

3.2.2 System Feature 2: Visual Language Model Processing

The system will use a Visual Language Model (VLM) to analyze the uploaded UI screenshots, extracting UI elements and their layout.

- The system must accurately identify all UI components such as text fields, checkboxes, radio buttons, and buttons.
- The system must provide the spatial relationships between identified components to maintain context.
- The model should extract both visual and textual information from the UI form.

3.2.3 System Feature 3: Test Case Generation

The system will automatically generate test cases based on the extracted UI elements from the Visual Language Model.

- The system must generate test cases covering all form fields (e.g., positive, negative, and boundary test cases for input fields).

- The system must handle test case generation for multi-step forms.
- The system must create a feature file or export test cases in a format compatible with test automation tools.

3.2.4 Test Case Execution via Automation Tools

The system will integrate with test automation tools like Selenium to execute the generated test cases.

- The system must export generated test cases in a format compatible with Selenium.
- The system must allow for the automated execution of test cases and return detailed results.
- The system must generate reports showing test case success/failure rates and detailed logs for any errors encountered during execution.

3.3 External Interface Requirements

This section outlines the external interfaces that the system will interact with, detailing the requirements for how the system will communicate with users, hardware, software, and other external systems.

3.3.1 User Interfaces

The system provides the interface to upload the image and show the generated test cases based on that uploaded UI image of the form.

3.3.2 Hardware Interfaces

A GPU-based server is recommended to handle Visual Language Model (VLM) tasks. Local storage is required on the client side to store and upload screenshots for test case generation.

3.3.3 Software Interfaces

Framework to load the VLMs into the system. Selenium API to execute the generated test case in the automated environment.

3.3.4 Communication Interfaces

HTTPS protocol is required for secure communication between the front-end interface (browser) and the server-side components, ensuring that UI screenshots are uploaded, and test cases are reviewed securely.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

The system must generate test cases quickly and efficiently, even with large forms and numerous test cases. It should handle multiple inputs simultaneously without significant performance loss, ensuring acceptable response times for test case generation and execution.

3.4.2 Safety Requirements

The system should validate input data to prevent errors and ensure correct test case generation. It must be resilient to edge cases and provide error detection and alerts to avoid faulty test cases that could cause system failures.

3.4.3 Security Requirements

Sensitive form data must be protected from unauthorized access. User access should be controlled with role-based authentication, and data exchanged with external tools should be encrypted to prevent security breaches.

3.4.4 Software Quality Attributes

The system should be reliable, ensuring consistent performance with minimal downtime. It must be easy to maintain and update, and offer a user-friendly interface for developers and testers to use efficiently with little training.

3.5 System Requirements

3.5.1 Database Requirements

The proposed system does not rely on a traditional relational or NoSQL database for its core functionality. Instead, it uses structured file-based storage and in-memory data structures to manage inputs, intermediate artifacts, and outputs. When a user uploads a UI screenshot or provides a URL, these assets are temporarily stored in the local file system for further processing and test generation. The system dynamically generates a test case table—derived from the visual layout of the uploaded image—which is maintained in memory during the session.

Feature files and step definition scripts generated by the language model are written to a designated workspace folder, enabling users to manually review

or reuse the test scripts. Although a persistent database is not essential for core operations, the architecture can be extended to include lightweight databases like SQLite or document-based systems such as MongoDB. This would enable long-term storage of test sessions, historical results, or user analytics if required in future versions. The system assumes the host environment supports basic file read/write permissions and has adequate local storage capacity to handle temporary data and generated outputs.

3.5.2 Software Requirements

The system is developed to run in a standard Python environment and does not require any proprietary software. It relies on commonly used open-source libraries and frameworks for web interface creation, browser automation, and test execution. A modern web browser is also required to support automated UI analysis tasks. The software is compatible with major operating systems including Windows, Linux, and macOS, provided the required packages are installed. Optional integration with AI models may require access to additional dependencies or APIs, depending on the deployment configuration. The software stack is lightweight and can be set up on a personal computer without enterprise-level infrastructure.

3.5.3 Hardware Requirements

The server must be occupied with a good amount of RAM and GPUs to process the data and run VLMs. End users can access the system via standard web browsers on desktops or laptops. No specific hardware requirements are imposed on the client side.

3.6 Analysis Models: SDLC Model to be applied

For the development of the proposed intelligent UI testing platform, we adopted the V-Model (Validation and Verification Model) of the Software Development Life Cycle (SDLC). The V-Model is a linear and sequential development methodology that places a strong emphasis on rigorous testing at every stage of the development process. It is especially well-suited for systems requiring reliability, traceability, and validation, such as test automation platforms.

3.6.1 Requirements Analysis Phase

In this phase, both functional and non-functional requirements were gathered. Functional requirements included:

- Uploading web page screenshots and URLs
- Automatic test case generation from visual UI
- Execution and result reporting via Behave framework

Non-functional requirements involved performance (test execution time), scalability (handling multiple pages), and usability (no-code interface for testers).

3.6.2 System Design Phase

The architecture was designed with modular components:

- Frontend (Streamlit-based interface)
- VLM and LLM model interfaces (LLaMA-3.2 Vision LLaMA-3.3 Versatile)
- Web crawler/XPath extractor (Selenium with ChromeDriver)
- Code generator module (for .feature and step files)
- Execution module (Behave integration)

Clear data flow and model interaction diagrams were created to support architectural decisions.

3.6.3 Module Design Phase

Each module was designed and tested independently:

- Prompt templates were designed for the VLM and LLM layers.
- The visual flow feature (joining multiple screenshots with arrows) was implemented to simulate user navigation.
- Test case table formatting and interactivity were developed in Streamlit.

3.6.4 Coding and Integration Phase

Development was carried out in Python with use of:

- selenium and chromedriver for DOM parsing
- OpenAI API-compatible interface for LLaMA models
- behave for execution and reporting

Version control and modularity were emphasized for easy integration and debugging.

3.6.5 Testing Phase (Verification)

Testing was performed at every stage as per the V-Model:

- Unit Testing: Each module (XPath extractor, code generator, test runner) was individually validated.
- Integration Testing: Interactions between the vision model, code generator, and test runner were tested using real web examples.
- System Testing: The full platform was evaluated on 5+ sample web UIs (login form, admin dashboard, e-commerce form, etc.)
- Validation: Test cases generated were reviewed against human-created test cases for coverage and correctness.

3.6.6 Deployment and Maintenance

The application was deployed in a local environment and packaged as a standalone Streamlit app. Provision was made for future extension—such as incorporating GPT-4V or supporting mobile-responsive UI testing.

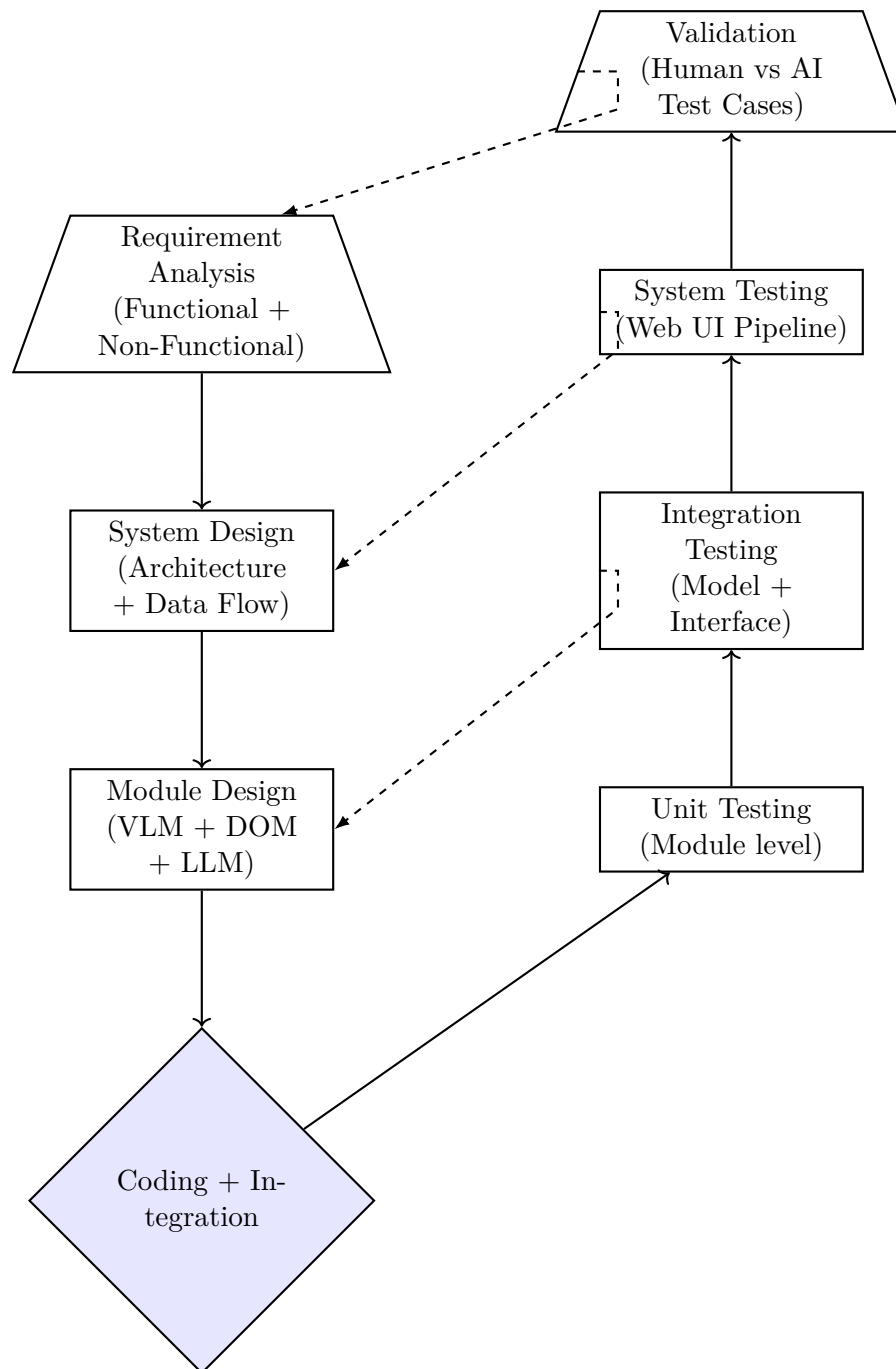


Figure 3.1: V-Model Applied to Intelligent UI Testing Platform Development

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

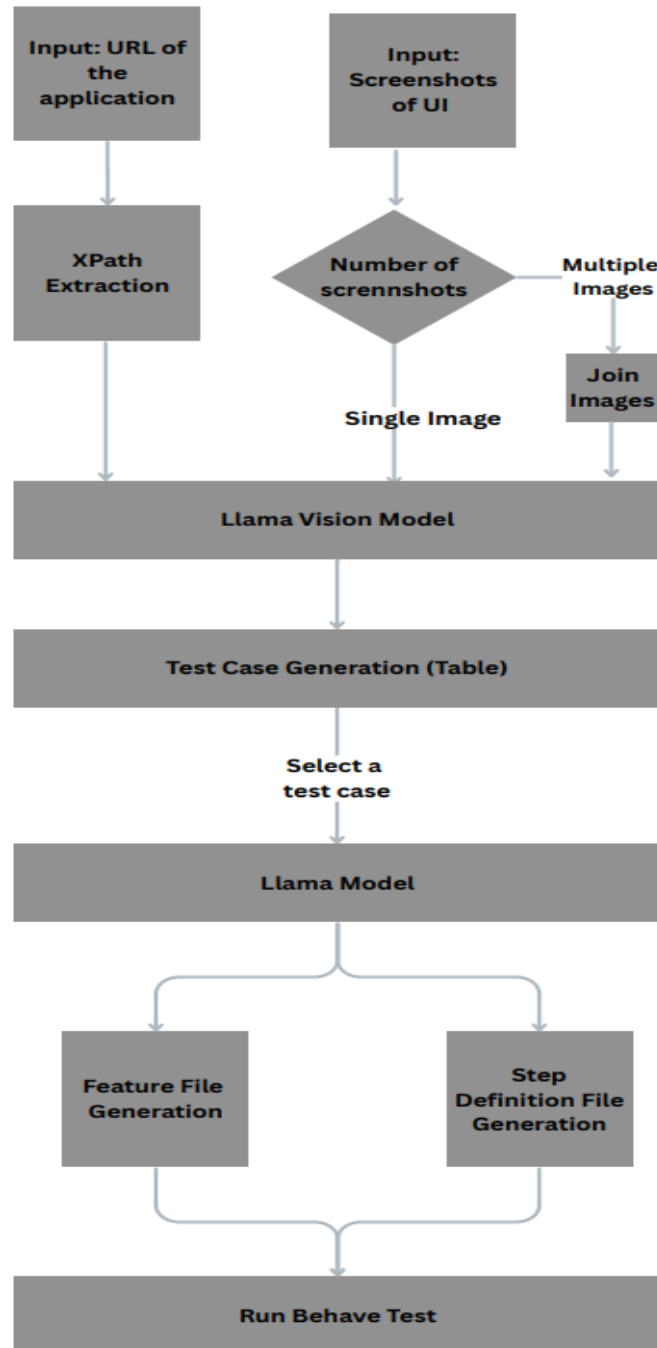


Figure 4.1: Architectural Diagram

4.2 Data Flow Diagrams

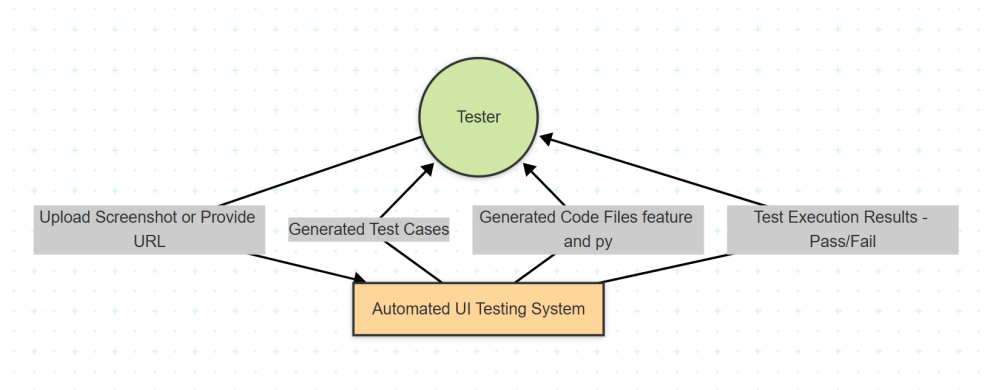


Figure 4.2: Level 0 Data Flow Diagram

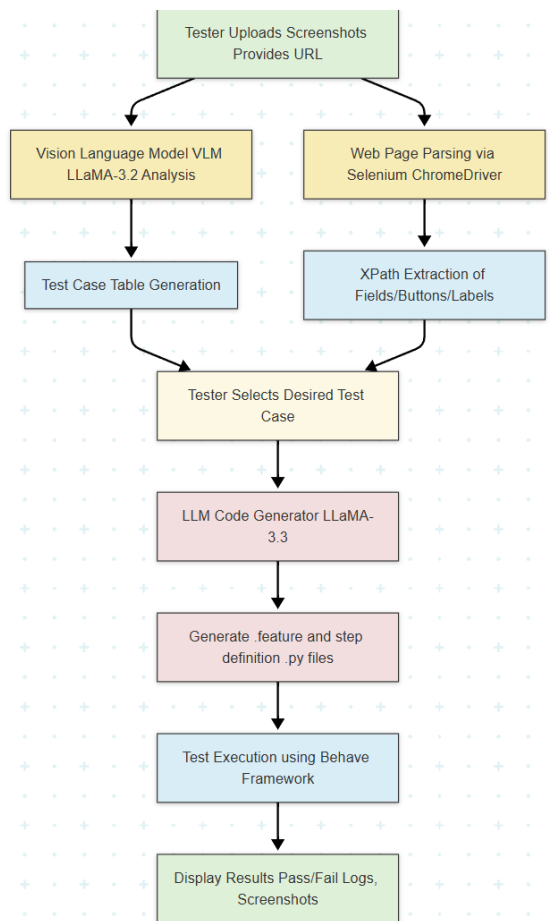


Figure 4.3: Level 1 Data Flow Diagram

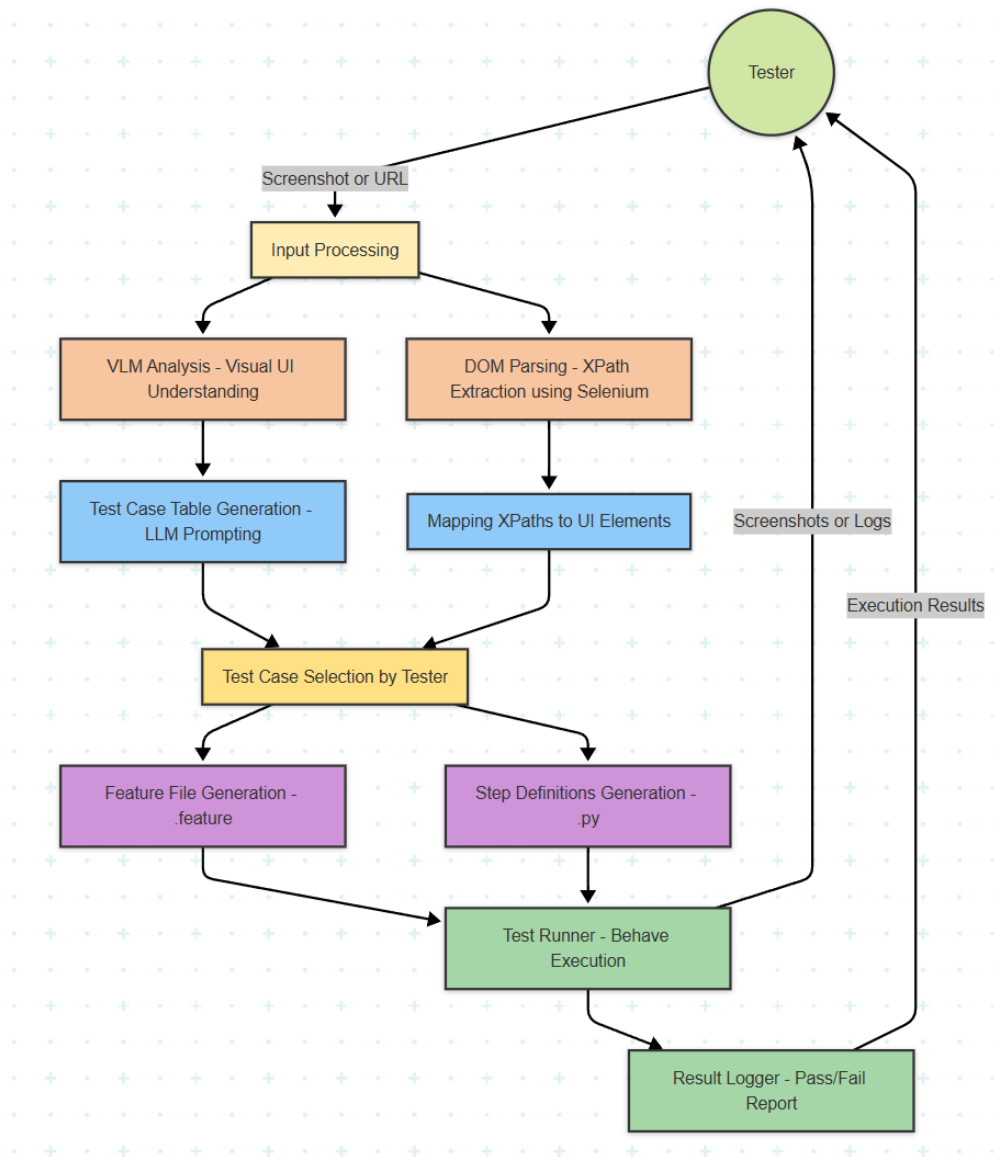


Figure 4.4: Level 2 Data Flow Diagram

4.3 UML Diagrams

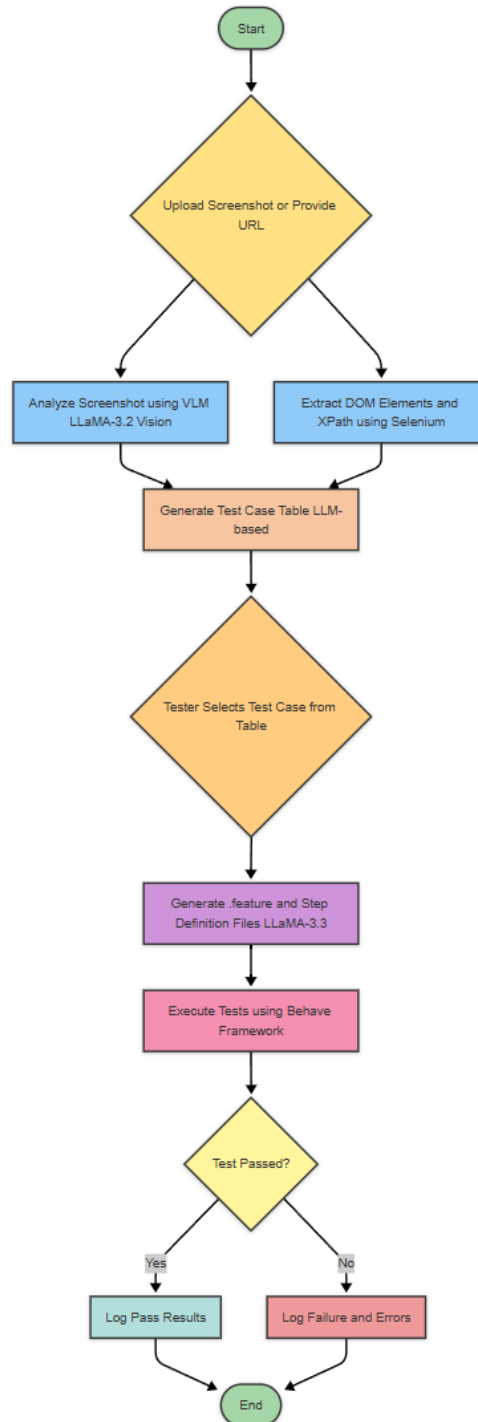


Figure 4.5: Activity Diagram

4.4 Sequence Diagram

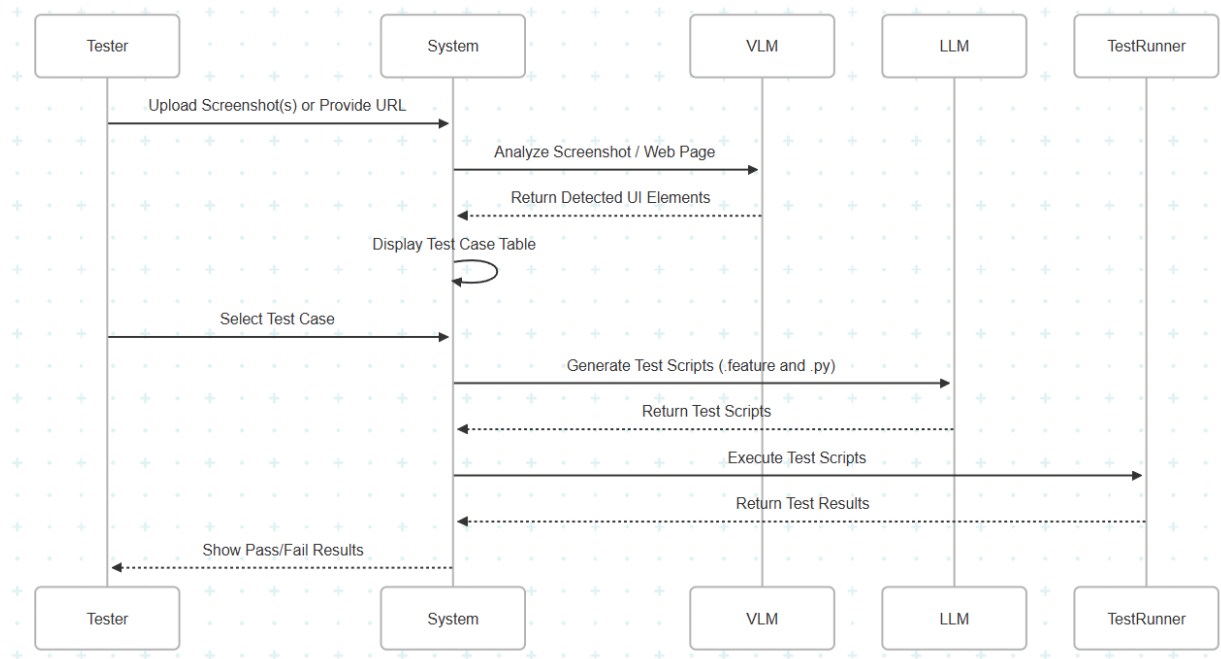


Figure 4.6: Sequence Diagram

CHAPTER 5

PROJECT PLAN

The project plan outlines the estimates, risks, schedule, and organizational structure for the development of project Automatic Testcase generation from UI images of form. Effective project planning ensures that the goals are met within the given timeline, while considering potential risks and ensuring proper team coordination.

5.1 Project Estimate

The initial estimates roughly calculate the time and resources required to complete the project. Key components include:

Time Estimation: Developing a system that uses multimodal LLMs for the generation of test cases from the UI image of the form will require approximately 5–6 months. The main phases include:

- Research and model selection: 2 months
- Proof of Concept generation and evaluation: 1.5 months
- Integration with other existing systems: 1–2 months
- Final testing and deployment: 1 month

Resource Estimation: The team will require:

- Access to LLMs (LLaVA, LLaMA, etc.)
- GPUs for loading the models and performing inference
- A dataset of UI forms for testing and evaluation

5.1.1 Reconciled Estimates

Once the initial estimates are reviewed, a more refined version—reconciled estimates— can be provided. These estimates are adjusted based on factors like risk analysis, team expertise, and budget constraints:

- The time required may increase by 10–15% if there are delays in data collection or evaluation of the result of PoC.
- Resource requirements might fluctuate depending on the availability of closed LLMs.
- Effort Estimates: Approximate time required for each phase of the project (e.g., research, development, model training, testing, and integration).

5.1.2 Project Resources

- Hardware: High-performance GPUs (NVIDIA A100) for training and fine-tuning LLaMA models, along with standard testing machines for execution.
- Software: Open-source tools like Streamlit for the UI, PyTorch for model inference, and Selenium/Behave for test automation.
- Datasets: Custom-collected UI screenshots from diverse web applications and publicly available datasets to train and validate the vision models.

5.2 Risk Management

- Risk Management

5.2.1 Risk Identification

Given below is a list the potential risks that could impact your project. These risks could be related to technical challenges, resource constraints, time delays, or other project- specific factors.

- Technical Risk: Difficulty in training the AI model to accurately process multi- screen forms.
- Data Availability: Lack of sufficient diverse screenshots of online payment forms to properly train the model.
- Integration Issues: Challenges in integrating the generative AI models with existing UI testing tools like Selenium.
- Resource Constraints: Limited access to high-performance computing resources needed for AI model training.
- Time Risk: Project delay due to unforeseen technical challenges or model inaccuracies
- Model Generalization Risk: The AI model may fail to generalize well across different form layouts and payment system interfaces.

5.2.2 Risk Analysis

We have evaluated the likelihood and potential impact of the identified risks.

- Technical Risk: High likelihood and medium impact. Complex image-based models may take time to develop and tune.
- Data Availability: Medium likelihood, but high impact. Insufficient data could lead to poor model performance.
- Integration Issues: Medium likelihood, medium impact. The complexity of integrating AI-generated test cases into a UI testing pipeline could slow development.
- Resource Constraints: Low likelihood, medium impact. Limited GPU or cloud resources might delay model training.

- Time Risk: Medium likelihood, high impact. Delays in development could impact delivery timelines.
- Model Generalization Risk: Medium likelihood, high impact. A model that struggles to generalize could affect the project's core functionality.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

We have outlined our plan for mitigating, monitoring, and managing each risk. This ensures that each identified risk has a strategy to either avoid it, reduce its impact, or handle it if it occurs.

- Technical Risk: Mitigation involves conducting frequent testing and prototyping to ensure the AI model is progressing as expected. Consider breaking down the problem into smaller tasks and using pre-trained models.
- Data Availability: Mitigation can be achieved by collecting a diverse set of screenshots or using data augmentation techniques to expand the dataset. Consider using synthetic data if real-world data is limited.
- Integration Issues: Plan to conduct early testing with Selenium and generative AI models to ensure smooth integration. Keep close collaboration between AI and testing teams.
- Resource Constraints: Monitor resource usage and plan for cloud-based solutions to scale up computational needs as required.
- Time Risk: Closely track the project's timeline and maintain a buffer for unexpected delays. Use agile methodologies to iterate and deliver parts of the project incrementally.
- Model Generalization Risk: Regularly test the AI model on diverse forms from various payment systems and adjust the model or training data as needed. This subsection shows that you have strategies in place to actively manage risks throughout the project.

5.3 Project Schedule

5.3.1 Project Task Set

- Task 1: Research and Requirement Gathering
- Task 2: Completion of Document of Understanding and Generation of Proof of Concept
- Task 3: Evaluation of the PoC
- Task 4: Updation of the system UI and delay module
- Task 5: Integration with other existing system
- Task 6: Testing and evaluation

5.3.2 Timeline Chart

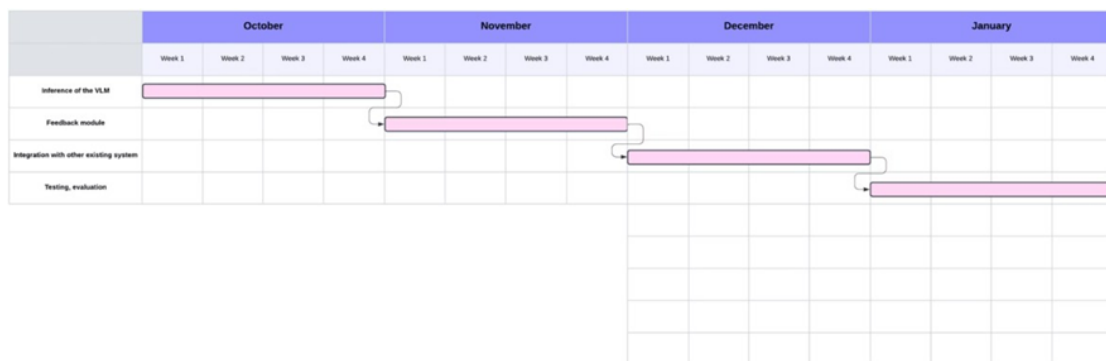


Figure 5.1: Timeline Chart

5.4 Team Organization

5.4.1 Team structure

The project team consists of the following roles:

- Project Manager: Oversees project execution, ensures deadlines are met, and manages communication.
- ML Expert: Develops the pipelines using multimodal LLMs.

- Web Developer: Develops the user interface that is appropriate for the system
- Testers: Conduct extensive testing to ensure system functionality and accuracy.

5.4.2 Management reporting and communication

Daily Standup call with Internal and External Guide from Sarvatra Technologies

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The system is modularized into the following components:

- **UI Input Module:** Accepts visual inputs in the form of screenshots/webpage URLs.
- **Vision-Language Model (VLM) Module:** Used to detect UI components (buttons, input fields, labels) and their hierarchical layout. Also extracts semantic context and generates a structured JSON representation of the UI.
- **Test Case Generation Module:** Leverages LLaMA-3 Versatile (LLM) to convert VLM output into human-readable test scenarios like Gherkin-style feature files. It also supports user customization including number of test cases, edge cases.
- **Script Generation Execution Module:** Translates test scenarios into executable scripts (Python + Behave) using ChromeDriver for dynamic XPath extraction.
- **User Interface (Streamlit):** Provides a web-based interface for uploading inputs, reviewing generated test cases, and monitoring execution results.

6.2 Tools and Technologies Used

- **AI Models:** LLaMA-3 90B Vision, LLaMA-3 70B Versatile
- **Testing Frameworks:** Behave, Selenium WebDriver, ChromeDriver
- **Backend:** Python (OpenCV, PyTorch)
- **Frontend:** Streamlit
- **Version Control:** Git, GitHub

6.3 Algorithm Details

6.3.1 Computer Vision (VLM)

- **Object Detection:** Identifies UI components using bounding boxes (e.g., buttons, text fields).

- OCR (Optical Character Recognition): Extracts text labels from UI elements (e.g., "Submit" button).
- Layout Parsing: Hierarchical segmentation of UI structures (e.g., forms, grids) using transformer-based models like LayoutLMv2.

6.3.2 Natural Language Processing (LLM)

- Few-shot Prompting: Guides LLaMA-3 to generate Gherkin-style test scenarios from VLM output.
- Constraint Inference: Deduces input validation rules (e.g., email format, mandatory fields) from contextual clues.

6.3.3 Test Script Synthesis

- XPath Generation: Dynamically creates locators for UI elements using ChromeDriver's DOM analysis.
- Code Generation: Transforms natural language test cases into executable Python scripts (Behave step definitions).

CHAPTER 7

SOFTWARE TESTING

Software testing plays a critical role in ensuring the functionality, accuracy, and robustness of the proposed automated UI testing platform. Since the system integrates visual understanding, natural language generation, and test execution, comprehensive testing is essential at both the component and integration levels.

7.1 Type of Testing

To ensure reliability, multiple levels of testing were applied throughout the development of the system, based on the V-Model SDLC. These include:

7.1.1 Unit Testing

Each core module was tested independently:

- XPath Extractor: Validated against known input fields and labels on public HTML forms.
- VLM Model Output Parser: Checked for accurate recognition of UI elements from screenshots.
- LLM Prompt Logic: Ensured consistent generation of structured test cases from extracted UI data.

7.1.2 Integration Testing

Tested the interaction between modules:

- Verified that the UI element extraction (VLM) correctly linked to the XPath generated from ChromeDriver.
- Ensured that the selected test case row led to valid .feature and .py code generation via LLM.
- Validated that code could be passed to the Behave runner without modification.

7.1.3 System Testing

Conducted on the end-to-end workflow:

- Full test cases were executed on multiple webpages including login forms, contact pages, and shopping carts.

- The system was evaluated on different browsers and screen resolutions.
- Emphasis was placed on handling multi-screenshot flows using the arrow-joining feature.

7.1.4 Validation Testing

The final results from the system were compared against:

- Manually written test cases by testers.
- Feature files generated using traditional automation tools. This comparison confirmed that the AI-generated test cases covered equivalent or broader functionality with minimal human input.

7.2 Test cases & Test Results

7.2.1 Test Case Relevance to Uploaded Screenshot

We evaluated the relevance of test cases generated by Pixel2Test by uploading screenshots from various popular websites. For each screenshot, 10 test cases were generated and manually assessed for contextual relevance. The results are summarized in Table [7.1](#).

Table 7.1: Relevance of Generated Test Cases

Website	Relevant Test Cases (out of 10)
OrangeHRM	9
Google	10
Mail	10
Facebook	10
YouTube	9
Flipkart	10
GitHub Login	10
Amazon	9
Gmail	9
LinkedIn Settings	7
Booking.com	9

7.2.2 Execution Success Rate and Required Modifications

We tested the executability of generated test cases on the OrangeHRM website. Table [7.2](#) shows the level of manual intervention required to execute each test case.

Table 7.2: Test Case Execution and Required Changes (OrangeHRM)

Test Case Title	Required Modification Level
Valid login credentials	No Change
Invalid username	Moderate Change
Invalid password	Moderate Change
Empty username and password	Moderate Change
Forgot password link	Minor Change
Username field is required	Moderate Change
Password field is required	Moderate Change
SQL injection attempt	Small Change

7.2.3 Reduction in Human Effort and Time

One of the key benefits of the *Pixel2Test* system is the drastic reduction in manual effort and time required for end-to-end test creation and execution. The average time taken by the system to complete the entire pipeline—from image upload to execution—is approximately **1 minute and 15 seconds** (± 15 seconds). In contrast, manual testing by human testers for the same process typically takes between **45 to 50 minutes**. This translates to a time efficiency improvement of over **95%**, highlighting the system’s potential for accelerating UI testing workflows in real-world scenarios.

CHAPTER 8

RESULTS

8.1 Outcomes

1. The Pixel2Test framework successfully demonstrated the feasibility of automating UI testing through AI-driven visual and language models. Preliminary testing on diverse web applications showed that the system could accurately identify UI components and generate executable test scripts.
2. The Streamlit interface provided an intuitive way for non-technical users to validate UI functionality, bridging the gap between design and testing.
3. Overall, the project validated the potential of AI to revolutionize UI testing, offering a scalable alternative to traditional methods while maintaining high accuracy and usability.

8.2 Screen Shots

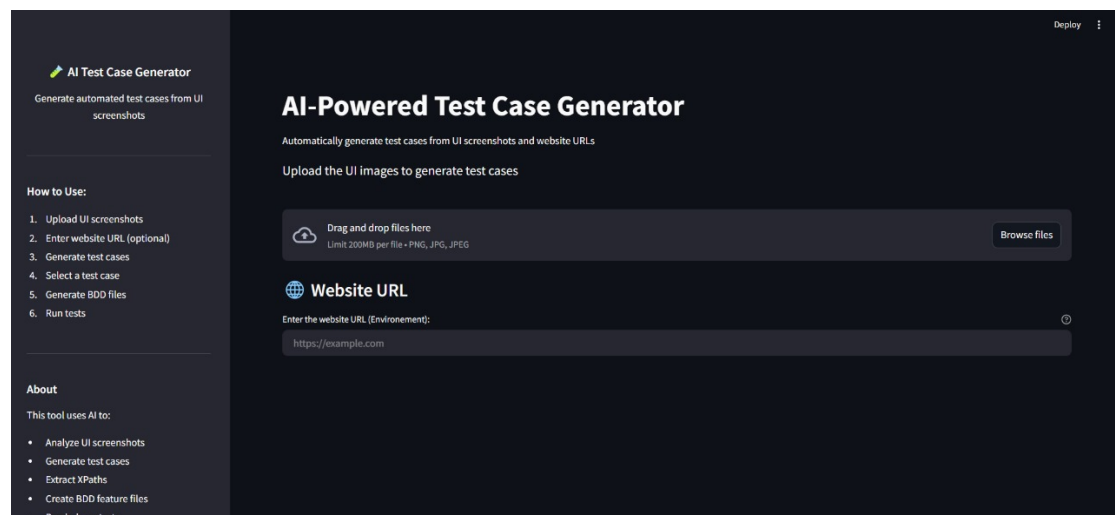


Figure 8.1: Streamlit UI

Test Case ID	Description of the Test Case	Steps to Perform the Test Case	Expected Result
TC-01	Valid Login Credentials	1. Enter valid username 'Admin' in the username field 2. Enter valid password 'admin123' in the password field 3. Click the 'Login' button	The user should be successfully logged in to the application
TC-02	Invalid Login Credentials	1. Enter invalid username 'Admin1' in the username field 2. Enter valid password 'admin123' in the password field 3. Click the 'Login' button	The user should see an error message indicating that the credentials are incorrect
TC-03	Empty Username Field	1. Leave the username field empty 2. Enter valid password 'admin123' in the password field 3. Click the 'Login' button	The user should see an error message indicating that the username is required
TC-04	Empty Password Field	1. Enter valid username 'Admin' in the username field 2. Leave the password field empty 3. Click the 'Login' button	The user should see an error message indicating that the password is required
TC-05	Forgot Password Link	1. Click on the 'Forgot your password?' link	The user should be redirected to a password recovery page

Figure 8.2: Generated Test Cases

Tests executed successfully!

Test Output:

```

Feature: Login to OrangeHRM Application # TC-01_feature_file.feature:1

Scenario: Valid Login Credentials # TC-01_feature_file.feature:2
  Given I am on the login page # steps/TC-01_step_definition.py:8
  When I enter valid username and password # steps/TC-01_step_definition.py:15
  And I click the Login button # steps/TC-01_step_definition.py:29
  Then I should be successfully logged in to the application # steps/TC-01_step_definition.py:36

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
4 steps passed, 0 failed, 0 skipped, 0 undefined
Took 0m31.786s
          
```

Figure 8.3: Execution Of Test Case

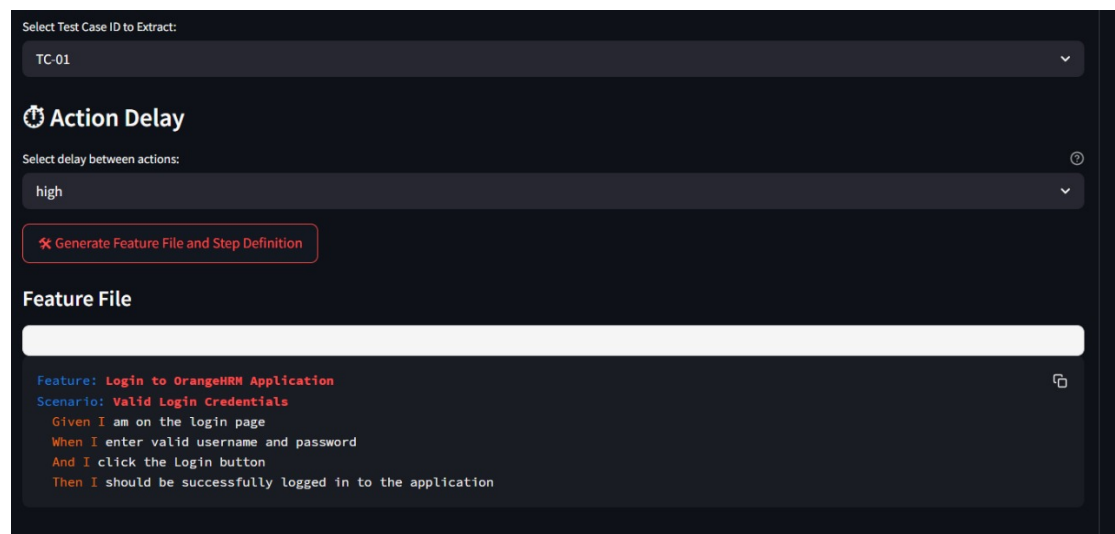


Figure 8.4: Time Delay Introduced

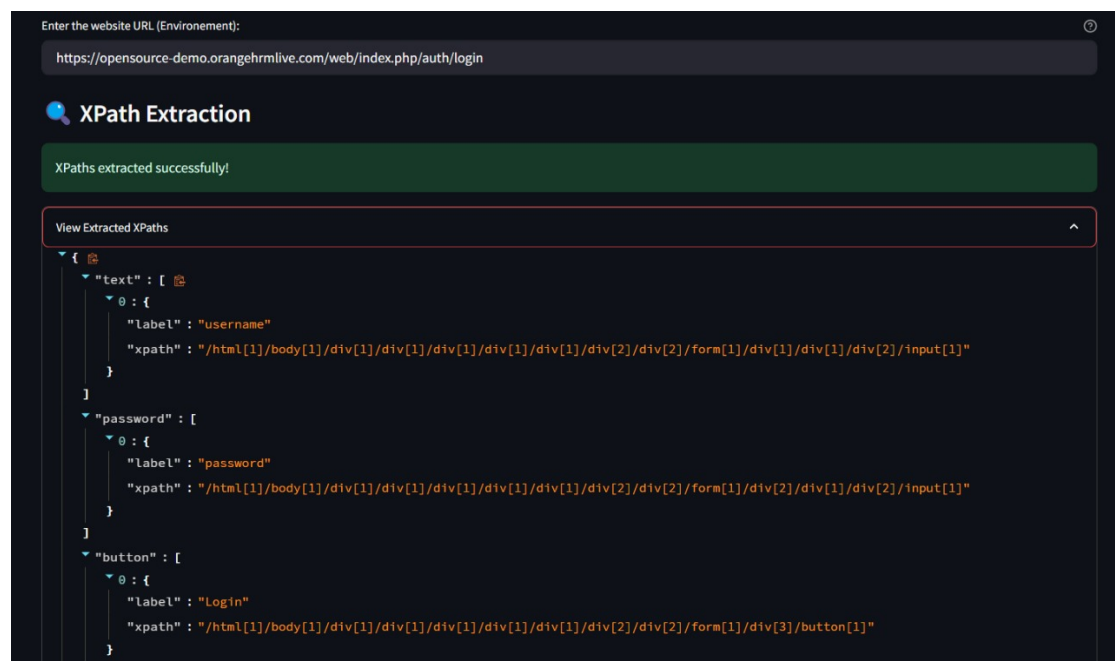


Figure 8.5: XPaths Extracted

CHAPTER 9

CONCLUSIONS

9.1 Conclusion

The project successfully demonstrates the feasibility of automating the generation of UI test cases using Visual Language Models (VLMs). By leveraging models such as LayoutLM and LLaVA, the system can efficiently process screenshots of online payment forms and generate exhaustive test cases. This automation significantly reduces manual effort, enhances testing coverage, and minimizes human errors in the testing process. The integration of test case execution via Selenium further streamlines the end-to-end testing process, allowing developers and testers to validate form functionality quickly and efficiently.

9.2 Future Work

1. Improvement in Model Accuracy
2. AI Model Generalization
3. Integration with Additional Testing Frameworks
4. Real-Time Feedback

9.3 Applications

1. UI Testing in Web Development: The system is highly useful in automated UI testing, particularly for form-heavy applications like e-commerce platforms, online surveys, and customer registration systems.
2. Software Quality Assurance: It can be integrated into the software quality assurance (QA) process to improve testing efficiency and ensure higher software reliability.
3. Regression Testing: The tool can be employed for regression testing, where forms need to be repeatedly tested after updates or bug fixes to ensure that new changes do not break existing functionality.
4. Compliance Testing: In industries that require compliance with regulatory standards (e.g., finance, healthcare), the system can help automate the testing of forms to ensure they meet necessary guidelines.

CHAPTER 10

APPENDIX

10.1 Appendix A

The problem of recognizing UI components from images and generating test cases can be broken down into subproblems that are individually solvable using image processing and AI-based models, such as Visual Language Models. The satisfiability of this problem is contingent on training data availability and model accuracy, making it a feasible task with sufficient resources.

The problem of recognizing UI components from images and generating test cases can be broken down into subproblems that are individually solvable using image processing and AI-based models, such as Visual Language Models. The satisfiability of this problem is contingent on training data availability and model accuracy, making it a feasible task with sufficient resources.

10.2 Appendix B

10.2.1 Survey Paper Details:

- **Paper Status:** Applied
- **Name of Journal:** ICTACT Journal on Data Science and Machine Learning
- **Title:** AUTOMATED TEST CASE GENERATION FROM UI IMAGES USING VISUAL LANGUAGE MODELS

10.2.2 Implementation Paper Details:

- **Paper Status:** Applied
- **Name of Conference:** International Conference on Business Intelligence, Computational Mathematics, and Data Analytics
- **Title:** Pixel2Test: Automated End-to-End UI Testing Using Visual Inputs and LLaMA Models

10.3 Appendix C

Plagiarism Report: Our report has 8% plagiarism, which was analyzed with the help of Ouriginal by Turnitin by our Project Guide.

CHAPTER 11

REFERENCES

- [1] Selenium Contributors, “Selenium WebDriver,” <https://www.selenium.dev>, accessed: 2024-04-15.
- [2] Cypress.io, “Fast, easy and reliable testing for anything that runs in a browser,” <https://www.cypress.io>, accessed: 2024-04-15.
- [3] Appium Contributors, “Automation for Apps,” <http://appium.io>, accessed: 2024-04-15.
- [4] Katalon LLC, “Katalon Studio,” <https://katalon.com>, accessed: 2024-04-15.
- [5] SmartBear Software, “TestComplete,” <https://smartbear.com/product/testcomplete>, accessed: 2024-04-15.
- [6] J. Shi and Y. Zhou, “Llm4test: Large language models for end-to-end test automation,” *arXiv preprint*, vol. arXiv:2209.09516, 2022. [Online]. Available: <https://arxiv.org/pdf/2209.09516.pdf>
- [7] Testim.io, “AI-powered test automation,” <https://www.testim.io>, accessed: 2024-04-15.
- [8] Mabl Inc., “Intelligent Test Automation,” <https://www.mabl.com>, accessed: 2024-04-15.
- [9] T. Chen, J. Zhang, and Y. Xu, “TestGPT: Leveraging GPT for Automated Test Case Generation,” in *Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1–12.
- [10] Diffblue Ltd, “AI for Java Unit Testing,” <https://www.diffblue.com>, accessed: 2024-04-15.
- [11] M. Böhme, E. O. Soremekun, and S. Chattopadhyay, “TESTAR: Automated GUI Testing,” in *Proc. ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, 2016, pp. 1–11.
- [12] RaiMan and SikuliX Team, “Automate anything you see on your screen,” <http://sikulix.com>, accessed: 2024-04-15.
- [13] EyeAutomate, “Image-based test automation,” <https://eyeautomate.com>, accessed: 2024-04-15.
- [14] AppliTools Inc., “Visual AI Testing,” <https://applitools.com>, accessed: 2024-04-15.

- [15] S. Wang, Y. Fan, X. Li, and Y. Zhang, “Leveraging large vision-language models for better automatic web gui testing,” in *Proceedings of the IEEE International Conference on Software Engineering and Artificial Intelligence*, 2024. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10795054>
- [16] A. R. V. Dam and L. Š. Serbina, “Descriptron: Automating taxonomic descriptions using ai,” *bioRxiv*, 2025. [Online]. Available: <https://www.biorxiv.org/content/10.1101/2025.01.07.631758.full.pdf>
- [17] C. Zhang, Y. Liu, and H. Yang, “Gpt-test: Generating ui test scripts from screenshots using large language models,” in *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023.
- [18] T. N. Nguyen and H. A. Nguyen, “Ai4test: An intelligent testing assistant with vision-language capabilities,” *arXiv preprint*, vol. arXiv:2311.08221, 2023. [Online]. Available: <https://arxiv.org/abs/2311.08221>
- [19] D. Ran, H. Wang, Z. Song, and M. Wu, “Guardian: A runtime framework for llm-based ui exploration,” in *ACM International Symposium on Software Testing and Analysis (ISSTA)*, 2024. [Online]. Available: <https://dezhi-ran.com/publication/issta24-guardian/issta24-guardian.pdf>
- [20] Y. Li, Y. Yang, and Y. Li, “Test-agent: A multimodal app automation testing framework based on the large language model,” in *IEEE Conference on Digital Twins and Parallel Intelligence*, 2024. [Online]. Available: <http://www.liyangyang.com/wp-content/uploads/2024/12/DTPi24-Agent-YangyangLi.pdf>
- [21] V. Garousi, N. Joy, A. B. Keleş, and S. Değirmenci, “Ai-powered test automation tools: A systematic review and empirical evaluation,” *arXiv preprint*, vol. arXiv:2409.00411, 2024. [Online]. Available: <https://arxiv.org/abs/2409.00411>
- [22] T. Li, C. Cui, L. Ma, D. Towey, Y. Xie, and R. Huang, “Leveraging large language models for automated web-form-test generation: An empirical study,” *arXiv preprint*, vol. arXiv:2405.09965, 2024. [Online]. Available: <https://arxiv.org/abs/2405.09965>
- [23] A. Isabella and E. Retna, “Study paper on test case generation for gui based testing,” *arXiv preprint*, 2012. [Online]. Available: <https://arxiv.org/abs/1202.4527>

- [24] F. Macchi, P. Rosin, J. M. Mervi, and L. Turchet, “Image-based approaches for automating gui testing of interactive web-based applications,” in *2021 28th Conference of Open Innovations Association (FRUCT)*, Moscow, Russia, 2021, pp. 278–285.
- [25] Z. Khaliq, S. U. Farooq, and D. A. Khan, “A deep learning-based automated framework for functional user interface testing,” *Information and Software Technology*, vol. 150, p. 106969, 2022.
- [26] Z. Liu *et al.*, “Vision-driven automated mobile gui testing via multi-modal large language model,” *arXiv preprint*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.03037>
- [27] F. Wang *et al.*, “Large language model driven automated software application testing,” Technical Disclosure Commons, 2024. [Online]. Available: https://www.tdcommons.org/dpubs_series/6815
- [28] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, “Layoutlm: Pre-training of text and layout for document image understanding,” in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 1192–1200.
- [29] G. Kim *et al.*, “Ocr-free document understanding transformer,” *arXiv preprint*, 2021. [Online]. Available: <https://arxiv.org/abs/2111.15664>
- [30] H. Liu, C. Li, Y. Li, and Y. J. Lee, “Improved baselines with visual instruction tuning,” *arXiv preprint*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.03744>
- [31] Y. Xu *et al.*, “Layoutlmv2: Multi-modal pre-training for visually-rich document understanding,” *arXiv preprint*, 2020. [Online]. Available: <https://arxiv.org/abs/2012.14740>
- [32] LLaVA Team, “Large Language and Vision Assistant,” <https://llava-vl.github.io>, accessed: 2024-04-15.
- [33] J. A. et al., “Flamingo: A Visual Language Model,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 1–15.
- [34] OpenAI, “GPT-4V(ision) System Card,” <https://openai.com/research/gpt-4v-system-card>, OpenAI, Tech. Rep., 2023.
- [35] Cucumber Ltd, “BDD Testing Framework,” <https://cucumber.io>, accessed: 2024-04-15.

- [36] Behave Developers, “Behavior-Driven Development in Python,” <https://behave.readthedocs.io>, accessed: 2024-04-15.
- [37] A. Panichella, S. M. Kifetew, and P. Tonella, “Generating BDD Scenarios from User Stories,” in *Proc. IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2021, pp. 1–12.
- [38] R. Takanen, M. V. Mäntylä, and J. Itkonen, “AI-Generated Step Definitions for BDD,” in *Proc. IEEE Int. Conf. Softw. Testing, Verif. Valid. (ICST)*, 2022, pp. 1–10.

Document Information

Analyzed document	be_Report.pdf (D210301226)
Submitted	2025-04-29 05:41:00 UTC+02:00
Submitted by	Uspawar
Submitter email	uspawar@pict.edu
Similarity	8%
Analysis address	uspawar.pict@analysis.orkund.com

Sources included in the report

W	URL: https://www.Bartleby.com/topics/sensitivity-analysis/41 Fetched: 2025-04-29 05:42:00	  1
W	URL: https://www.coursehero.com/file/83472522/PROJECT-REPORT-1pdf/ Fetched: 2025-04-29 05:41:00	  1
W	URL: https://kbtcoe.org/wp-content/uploads/2016/12/REPORT-TEMPLATES-STAGE-2-2021-22.pdf Fetched: 2025-04-29 05:42:00	  1
W	URL: https://WWW.coursehero.com/file/236179213/BEproject-2nd-sem-report-3pdf/ Fetched: 2025-04-29 05:42:00	  1
W	URL: https://aischolar.org/attendees/index/EAUFXLQ Fetched: 2025-04-29 05:41:00	  1
W	URL: https://WWW.coursehero.com/file/236179261/BlckBookReport-Finalpdf/ Fetched: 2025-04-29 05:42:00	  1
W	URL: https://ijsrem.com/download/supply-chain-management-in-agriculture-using-blockchain/?refresh=677e982cd03f11736349740&wpdmdl=24880 Fetched: 2025-04-29 05:42:00	  2
W	URL: https://www.coursehero.com/file/12977115/Chapter-15-Information-Security/ Fetched: 2025-04-29 05:42:00	  1
W	URL: https://www.sentra.io/learn Fetched: 2025-04-29 05:42:00	  1
W	URL: https://fi.player.fm/podcasts/Angular-Air Fetched: 2025-04-29 05:41:00	  1

Acknowledgement for paper submission-reg Inbox x



ICT Academy <ijdml@ictacademy.in>

Feb 3, 2025, 12:00 PM



to me, adityadhodade@gmail.com, vedant.rawat777@gmail.com, siddhesh.dhadse@gmail.com, nikhil.karkhanis@sarvatra.in, uspawar@pict.edu

Dear Author,

Thank you for submitting the paper titled **"AUTOMATED TEST CASE GENERATION FROM UI IMAGES USING VISUAL LANGUAGE MODELS"** for possible publication in **ICTACT** Journal on **Data Science and Machine Learning**.

The paper has been assigned an ID: **IJDSML0903**. Please quote this ID. No. in all the future communications. The paper will undergo plagiarism checking process and then peer blind review process.

The submission of the article is taken as an implicit guarantee that the work is original, that it is author(s) own work, that the results have not already been published nor considered and will not be offered elsewhere while under consideration for an **ICTACT** Journal.

Kindly fill the detailed Author's information form (mandatory) and send at the earliest.

We again thank the author and co-authors for considering **ICTACT** Journal. We hope you submit few more articles of your research work to **ICTACT** journals in future.

The complete technical evaluation procedure takes about 12 to 16 weeks. We won't reply to any emails inside the specified time frame.

We would like to inform you that going forward, emails only (ijdml@ictacademy.in) will be accepted for all correspondence. No phone calls are accepted by us.

Regards

ICTACT Journals Section

AUTOMATED TEST CASE GENERATION FROM UI IMAGES USING VISUAL LANGUAGE MODELS

Aditya Dhodade¹, Prasad Dhondge², Vedant Rawat³, Siddhesh Dhadse⁴, Nikhil Shankar Karkhanis⁵ and Prof. Urmila Pawar⁶

¹Computer Engineering, Pune Institute of Computer Technology, India

E-mail: adityadhodade@gmail.com

²Computer Engineering, Pune Institute of Computer Technology, India

E-mail: prasaddhondge1007@gmail.com

³Computer Engineering, Pune Institute of Computer Technology, India

E-mail: vedant.rawat777@gmail.com

⁴Computer Engineering, Pune Institute of Computer Technology, India

E-mail: siddhesh.dhadse@gmail.com

⁵Head – Quality Process, Sarvatra Technologies, India

E-mail: nikhil.karkhanis@sarvatra.in

⁶Computer Engineering, Pune Institute of Computer Technology, India

E-mail: uspawar@pict.edu

Abstract

Software testing is a crucial part of the development cycle, ensuring that applications meet user expectations and function without errors. However, generating exhaustive test cases is a time-consuming and often manual process, especially when it involves forms with complex structures and varied input fields. Traditional methods of test case generation, particularly for form-based applications, rely on manual interpretation of form layouts and input fields which is prone to human error and limits the number of test cases that can be generated within given time constraints. OCR (Optical Character Recognition) methods which are used to extract text from form images struggle with document structure and layout. Although some machine learning models assist in the testing process, they lack the ability to automatically understand the complex interactions between the form's layout and input fields. With advancements in artificial intelligence (AI) and large language models (LLMs), there is an opportunity to automate this process using multimodal generative models. Our approach uses different generative models to analyze the subject to be tested and generate the test cases to find the loophole in the problem. Further, this test cases can be tested automatically. This approach significantly improves test coverage, reduces manual effort, and accelerates the testing process, ensuring robust software performance with minimal human intervention.

Keywords:

generative models, automated testing, test case generation

1. INTRODUCTION

User Interface (UI) Testing is important in ensuring that software applications provide flawless and user-friendly experience. The task of UI testing is to verify that the visual elements of a software application function correctly and satisfy user expectations. This includes interactions with forms, buttons, links, and other components to make sure that application looks appealing and function as expected across various devices and environments.

With rapid growth of digital platforms, the demand for robust, intuitive, and reliable user interfaces is very high today. Errors or discrepancies in the UI can lead to user dissatisfaction, poor usability, and even financial losses for

businesses. With the help of UI testing, we can ensure that all the interactive elements are performing correctly and thus in turn contributing to seamless user experience.

We need effective UI testing not only for aesthetics but to also ensure accessibility, functionality, and overall performance. Various testing elements like form validations, responsiveness, and visual accuracy across different browsers and devices is of utmost importance for delivering high-quality software. If we do not carry out thorough UI testing, software applications risk alienating customers owing to poor functioning, resulting in greater bounce rates and decreased customer satisfaction.

Traditionally, UI testing has been carried out manually where testers create test cases to verify that the interface meets requirements. But it is time-consuming, labor-intensive, and less accurate. This method cannot cope with the modern fast-paced development cycle of application. To address the inefficiency in manual testing, automated testing tools such as Selenium, TestComplete, etc. were introduced. But these tools allow the execution of predefined test cases only and struggle with the complex, dynamic UI. Although some machine learning models assist in the testing process, they lack the ability to automatically understand the complex interactions between the form's layout and input fields. However, this issue can be resolved by using generative AI models. With advancements in large language models (LLMs), there is an opportunity to automate this process by using vision-language models.

In this paper, we propose an approach that uses Visual language models (VLMs) for feature file generation. By inputting the images of the UI form, the VLMs can analyze the form structure, identify the elements present in it and generate quality assurance test cases. This method eliminates the need to maintain predefined test cases or put manual effort into creating new diverse test cases. By using VLMs, we can combine the power of the computer vision model with the

language model for a deeper understanding of UI. This enables the system to take both i.e., structural and semantic understanding of UI form, ensuring more diverse and edge test case generation. This approach improves both speed and accuracy than traditional methods of test case generation.

2. RELATED WORK

2.1 APPROACHES

In [1] authors have discussed some techniques used for GUI test case generation. In the first technique they have used feedback from the execution of an initial or "seed" test suite on an Application Under Test (AUT). It begins by creating an Event Interaction Graph (EIG), which models two-way interactions between GUI events. These test cases are executed, and the GUI's run-time state is recorded. Based on this data, an Event Semantic Interaction (ESI) relationship is identified, leading to the creation of an Event Semantic Interaction Graph (ESIG). This graph is used to generate more complex test cases for multi-way interactions (2-, 3-, 4-way), ensuring broader event coverage and detecting more faults. While this method increases fault detection due to varied event sequences, it can lead to a large number of test cases and may struggle with applications that have complex underlying business logic. Additionally, the process is not fully automated, which limits its scalability. The second method was using covering array technique which focused on maximizing event interaction coverage in GUI testing. A covering array (CA), denoted as $CA(N; t, k, v)$, is a matrix where every $N \times t$ subarray contains all possible combinations of t events (or interactions) at least once. Initially, an Event Interaction Graph (EIG) of the application is created to model how GUI events interact. This EIG is then partitioned into groups of interacting events, and constraints are identified. Using these groups, an abstract model is created for testing. The technique samples from the covering array to generate long test sequences that maintain t -way interaction coverage. These sequences are executed, and if some event interactions are missed, the process repeats to refine the coverage. In this method however, partitioning events and identifying constraints must be done manually, which limits its scalability and makes it resource-intensive for large applications. The third technique is about feedback-based approach designed to generate test cases with higher event interaction coverage and fewer infeasible test cases for GUI applications. The process begins with smoke tests, which act as seed tests to explore the GUI. From these, Event Semantic Interaction (ESI) relationships are generated, leading to the creation of an Event Semantic Interaction Graph (ESIG). Using techniques from combinatorial interaction testing, test cases are generated and executed in batches. After execution, code coverage is analyzed, and any infeasible test cases are identified and discarded. The GUI interaction model is updated, and a new batch of test cases is generated to cover previously uncovered interactions. This cycle repeats until all interactions are covered. However, it does not incorporate event contexts and lacks test adequacy criteria to measure fault detection effectiveness. In the last technique they focus on fixing infeasible test cases and improving test suite coverage for GUI

applications. Initially, a set of test cases is executed. If some are infeasible a genetic algorithm is applied to repair them. It works iteratively by removing infeasible test cases and generating new, feasible ones. The process starts with inputs like Event Flow Graphs (EFG), Event Interaction Graphs (EIG), and GUI structures, which model the interactions between GUI events. A covering array is used to create an initial set of event sequences, and these are assembled into test cases. The genetic algorithm iteratively adds the best test cases to the final suite, ensuring improved coverage without growing the test suite excessively. This method generates more efficient test suites, providing coverage for long sequences and reducing the number of infeasible test cases. However, it can be slow due to the high execution time, and the method is less scalable too.

In [2] researchers have introduced a framework that combines Machine Learning and Computer Vision algorithms to automate GUI testing by analyzing screenshots for Pattern Detection, Template Detection, and Text Recognition. Integrating with tools like Selenium and Tesseract OCR, it effectively detects visual changes and verifies GUI properties, handling interactive and animated interfaces well. However, the approach can be computationally intensive and depends on the quality of screenshots, requiring fine-tuning. Overall, it offers a comprehensive solution for testing complex GUIs beyond traditional methods.

In [3] authors have used transformers to automate the process of testcase generation directly from UI element description instead of relying on the test specification document from which test cases are extracted manually. They have used object detection algorithms EfficientDet and DETECTIONTransformer for detecting the elements from an application UI automatically and Tesseract to automatically identify the text present on the UI elements. Then they have used transformers like GPT-2 and T5 to transform the generated UI element description to actual test cases.

In [4] they have introduced VisionDroid, a novel approach for detecting non-crash functional bugs in mobile apps using a Multimodal Large Language Model (MLLM). VisionDroid generates "vision prompts" by aligning extracted text information with GUI screenshots, enabling the MLLM to comprehend the app's visual and textual context. It employs a function-aware explorer to navigate GUI pages deeply and a logic-aware bug detector to segment the exploration history into cohesive parts, identifying potential bugs by comparing GUI transitions with the intended logic. This approach effectively improves recall and precision in bug detection. It leverages MLLM's understanding of visual and textual data to adapt to diverse testing scenarios. However, VisionDroid's effectiveness depends on the MLLM's capabilities and incurs high computational costs due to its comprehensive exploration process, making it well-suited for complex, non-crash bug detection where visual cues are essential.

In [5] authors have given a large language model (LLM)-driven approach for automating software application testing, focusing on UI and A/B testing. It uses historical error data and prompt engineering to analyze screenshots of the

application under test, detecting UI rendering issues and generating test assertions automatically. By integrating LLM responses into a testing framework, it simulates manual QA testing and helps in identifying discrepancies between different versions of an application. This method enhances scalability, reduces manual testing efforts, cuts costs, and expands test coverage. However, its effectiveness depends on the quality of prompts and historical data, and it may be less efficient in identifying rare or specific UI errors not represented in the dataset.

2.2 VISUAL LANGUAGE MODELS

In [6] authors introduce LayoutLM, a pre-training model that integrates both text and layout information for document image understanding tasks. By extending the BERT model with 2D positional embeddings (to capture spatial layouts) and image embeddings (for visual features), LayoutLM effectively combines text and layout data. It is trained using a masked visual-language model and multi-label document classification tasks, outperforming state-of-the-art methods in form understanding, receipt understanding, and document classification. While it offers superior performance, it requires substantial computational resources and large-scale datasets for optimal results.

In [7] authors introduce us to "Donut", an OCR-free visual document understanding (VDU) model based on a Transformer architecture that processes document images directly without relying on OCR. It maps images to structured outputs like JSON through a visual encoder and textual decoder, addressing the limitations of traditional OCR-based methods, such as high computational costs, error propagation, and inflexibility with different languages. Donut is trained using a pre-train-and-fine-tune approach, showing state-of-the-art performance in tasks like document classification and information extraction. However, it requires large-scale pre-training and can struggle with tiny text elements in some images.

In [8] LLaVA framework is refined through two key improvements: integrating an MLP projection layer for better vision-language alignment and incorporating academic-task-oriented datasets like VQA for training. These updates boost data efficiency and enable handling both short and long-form answers, resulting in state-of-the-art performance across multiple benchmarks. The model scales efficiently to higher resolutions by splitting images into grids, improving visual perception and reducing hallucinations. While highly effective, the approach incurs higher computational costs and sometimes faces challenges in balancing response formats. It combines CLIP ViT-L/14 vision encoder and Vicuna for general-purpose visual and language understanding.

Table.1. Comparison of VLMs

Feature s	LayoutLM	Donut	LLaVA
Architecture	Transformer-based model with pre-training on text, layout, and image embeddings	Encoder-decoder transformer without OCR dependency	Combines Llama and CLIP-like vision encoders
Strengths	Strong layout awareness in documents, well-suited for structured document understanding	Removes the need for OCR, can directly generate structured text from document images	Highly capable of integrating image and language interaction powerful in visual
Weaknesses	Requires accurate layout information for full performance, may not generalize well to noisy data	Limited performance on complex documents with challenging layouts, especially for data	May struggle with complex visual reasoning tasks requiring deep world knowledge
Accuracy	SROIE: 95.24% F1 (Key information extraction) FUNSD: 84.20% F1 (Document understanding)	SROIE: 96.08% F1 (Receipt parsing) CORD: 89.32% F1 (Receipt information extraction)	~73.4% (on visual Q&A tasks such as Science QA, evaluated on reasoning-based tasks)

2.3 EXISTING SOFTWARE FOR UI TESTING

2.3.1 Selenium:

It is a widely used open-source framework for automating web application testing. Using Selenium testers create scripts that simulate user interactions with web browsers, enabling automated functional testing of web applications across different browsers and platforms. Major browsers like Chrome, Firefox, Edge and Safari. Test scripts can be written in multiple programming languages, such as Java, Python, C#, and JavaScript. Selenium integrates with tools like TestNG, JUnit, and Maven for managing test cases and running them efficiently. Selenium supports parallel test execution, allowing multiple tests to run concurrently.

2.3.2 TestComplete:

TestComplete is an automated testing tool that allows testers to create and run tests across desktop, web, and mobile applications. Multiple scripting languages like JavaScript,

Python, VBScript, etc. are supported by TestComplete. It also supports script less, keyword-driven approach which also makes it accessible to non-technical users. It offers features like record-and-playback capabilities, extensive test reporting, and support for data-driven testing, enabling thorough validation of applications. It seamlessly integrates into software development life cycle.

2.3.3 HPALM:

HPALM is HP Application Lifecycle Management software. It has a mechanism which can accurately locate elements on web pages. It supports a wide range of locators like XPath, CSS selector, etc. HPALM can record the interaction with the web application and generate the corresponding test script. Through this, we can execute the same script for the different sets of data.

3. RESEARCH GAP

Most of the existing automated UI testing software requires manual scripting of the test cases. Also, these test cases need to be created manually which may be prone to human error. Although this software can automate the test execution, the initial creation of test cases remains time-consuming. This test case creation becomes challenging when UI becomes dynamic and complex. Additionally, this software cannot generate test cases from UI images. They are restricted to using predefined scripts and cannot adapt to dynamic changes in the UI. This highlights a critical gap in automated UI testing, as there is no solution that fully automates the test case creation process using visual analysis.

4. PROPOSED METHODOLOGY

The proposed method focuses on generating feature files from UI form images using visual language models (VLM). The first step of this process is to take the input of image of form which is then passed to the visual language model. We can employ VLMs like LLaVA, LayoutLM, Donut, etc. which can understand both the visual structure and the semantic context of UI images. Once the image is processed by VLM, the model identifies the elements of the form like the input field, buttons, checkboxes, labels, and the constraints on the input field. It also understands the form structure by understanding the arrangement of the element and thereby the semantic meaning.

Following the extraction of all the information, it is used to generate feature files. These feature files are used to stack different test scenarios associated with each UI component. For example, feature file may contain scenarios to check for valid and input inputs, button functionality might be checked for proper response to user actions and so on. This automated process eliminates the need to manually craft the test case which significantly reduces the overall time. It also ensures diverse and edge test case scenarios.



Fig.1. Flow of the proposed methods

4.1 PROMPT FOR THE MODEL

The prompt for this purpose can be "You are a QA tester. Your task is to analyze the image of form, understand its structure and semantics and write few 'Test cases' in JSON format for this image. Each test case should include the following keys: 'Test Case ID', 'Description of the Test Case', 'Steps to perform the Test Case' and 'Expected Result'. The test case should be relevant to the type of the image of form given as input."

5. CONCLUSION

The proposed methodology presents an approach to automate the test case generation for UI forms using visual language models. By utilizing VLMs, the process eliminates the need for the manual creation of test cases using UI of image. This approach addresses the limitation of existing tools which rely on manual intervention for test case creation. Ultimately, this methodology contributes to more streamlined and robust UI testing practices.

REFERENCES

- [1] Isabella, A. and Retna, E. (2012) Study paper on test case generation for gui based testing, *International Journal of Software Engineering and Applications (IJSEA)*, Vol. 3, No. 1, Pp. 139-147.
- [2] F. Macchi, P. Rosin, J. M. Mervi and L. Turchet, "Image-based Approaches for Automating GUI Testing of Interactive Web-based Applications," *2021 28th Conference of Open Innovations Association (FRUCT)*, Moscow, Russia, 2021, pp. 278-285, doi: 10.23919/FRUCT50888.2021.9347592.
- [3] Z. Khaliq, S. U. Farooq, and D. A. Khan, "A deep learning-based automated framework for functional User Interface testing," *Information and Software Technology*, vol. 150, p. 106969, Jun. 2022, doi: 10.1016/j.infsof.2022.106969.
- [4] Z. Liu et al., "Vision-driven automated Mobile GUI testing via multimodal large language model," *arXiv (Cornell University)*, Jul. 2024, doi: 10.48550/arxiv.2407.03037.
- [5] Wang, Fei, et al. "Large Language Model Driven Automated Software Application Testing." *Technical Disclosure Commons*, (March 26, 2024) https://www.tdcommons.org/dpubs_series/6815 (2024).
- [6] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of Text and Layout for

Document Image Understanding,” *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1192–1200, Aug. 2020, doi: <https://doi.org/10.1145/3394486.3403172>

- [7] Kim, Geewook, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. "Ocr-free document understanding transformer." In *European Conference on Computer Vision*, pp. 498-517. Cham: Springer Nature Switzerland, 2022.
- [8] Y. Xu, Y. Xu, T. Lv, L. Cui, F. Wei, G. Wang, Y. Lu, D.

Florencio, C. Zhang, W. Che et al., “Layoutlmv2: Multimodal pre-training for visually-rich document understanding,” in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2021, pp. 2579–2591.

- [9] Liu, Haotian, Chunyuan Li, Yuheng Li, and Yong Jae Lee. "Improved baselines with visual instruction tuning." In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 26296-26306. 2024.
-